

操作系统

第二章 进程管理

第二部分 (4) 异常与外设中断

同济大学计算机系



Part 1 异常

- Unix V6++ 的异常处理过程
- 一个有趣的例子

一、基本概念

- 异常 (Exception) , 是CPU芯片发给自己的中断信号
- CPU无法顺利完成当前操作, 会给自己发异常信号, 激活内核排查错误、处理异常事件。

产生异常的原因	具体的语义	默认处理方式
应用程序有错	执行非法指令, 运算溢出, 访问非法内存	用信号杀死出错的现运行进程
需要为现运行进程扩展内存	本条指令访问的数据, 或下条要执行的指令不在内存	注1
内核有错	-	panic, 系统停摆, 等待管理员关机、修复、重启
硬件故障	-	用信号杀死现运行进程 (前, 用户态) 或 panic (前, 核心态)

注1: 虚拟存储器, 请求调页。连续存储管理方式的 Unix V6++ , 堆栈自动扩展

注2: 这张表不包含3#断点异常



二、i386芯片抛出异常（典型场景和Unix的处理方式 1）

#	Exception
0	Divide error
1	Debug
2	NMI
3	Breakpoint
4	Overflow
5	Bounds check
6	Invalid opcode
7	Device not available
8	Double fault
9	Coprocessor segment overrun
10	Invalid TSS
11	Segment not present
12	Stack exception
13	General protection
14	Page Fault
15	Intel reserved
16	Floating-point error
17	Alignment check
18	Machine check
19	SIMD floating point

现运行进程执行C语句 $c = a/b$ & $b == 0$

CPU抛出 0#异常(Divide Error), 表示除法出错。

Unix V6++执行0#异常处理程序:

- 向现运行进程发SIGFPE信号 (8#信号)
- 立即处理这个信号
 - `Exit()`杀死现运行进程 (除非应用程序有定义SIGFPE信号处理函数)

i386芯片抛出异常 (典型场景和Unix的处理方式 2)

#	Exception
0	Divide error
1	Debug
2	NMI
3	Breakpoint
4	Overflow
5	Bounds check
6	Invalid opcode
7	Device not available
8	Double fault
9	Coprocessor segment overrun
10	Invalid TSS
11	Segment not present
12	Stack exception
13	General protection
14	Page Fault
15	Intel reserved
16	Floating-point error
17	Alignment check
18	Machine check
19	SIMD floating point

MMU页级单元地址映射和内存保护操作**buggy**, 给不出操作数或下条指令的物理地址。

- CPU
- 1) 生成error code, 标识究竟哪儿错了
 - 2) 产生异常的当前虚地址VA → CR2寄存器
 - 3) 抛出 14#异常(Page Fault), 表示缺页

Unix V6++执行14#异常处理程序:

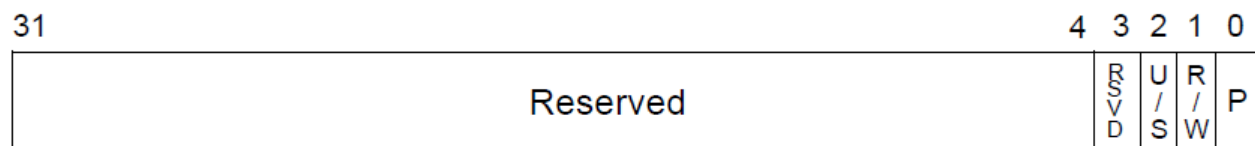
- 内核bug? Y, Panic系统停摆
- 需要为应用程序扩展用户栈? Y, 执行SStack()扩展堆栈
- 应用程序非法内存访问? 需要终止现运行进程
 - 向现运行进程发SIGFPE信号 (8#信号)
 - 立即处理这个信号
 - Exit()杀死buggy的现运行进程

异常处理结束后, 如果进程没有没杀死, IRET回用户空间, 重新执行引发异常指令, 会成功。

Unix V6++, 连续内存管理方式。现运行进程代码和可交换部分全部在内存。

Q1: 现运行进程执行时, 有可能缺页异常吗? 什么时候?

error code



- P**
- 0 The fault was caused by a nonpresent page. **PTE为空, 野指针 或 堆栈自动扩展**
 - 1 The fault was caused by a page-level protection violation. **访问权限不对**
- W/R**
- 0 The access causing the fault was a read.
 - 1 The access causing the fault was a write. **写只读页面**
- U/S**
- 0 The access causing the fault originated when the processor was executing in supervisor mode.
 - 1 The access causing the fault originated when the processor **CPU用户态执行时, 访问核心空间** was executing in user mode.
- RSVD**
- 0 The fault was not caused by a reserved bit violation.
 - 1 The page fault occurred because a 1 was detected in one of the reserved bit positions of a page table entry or directory entry that was marked present.

Figure 5-7. Page-Fault Error Code



三、i386的异常

0~31号中断， 0~31号异常

部分异常，硬件会产生error code

3种类型的异常：

Fault：有可能修复。修复后，重新执行故障指令。

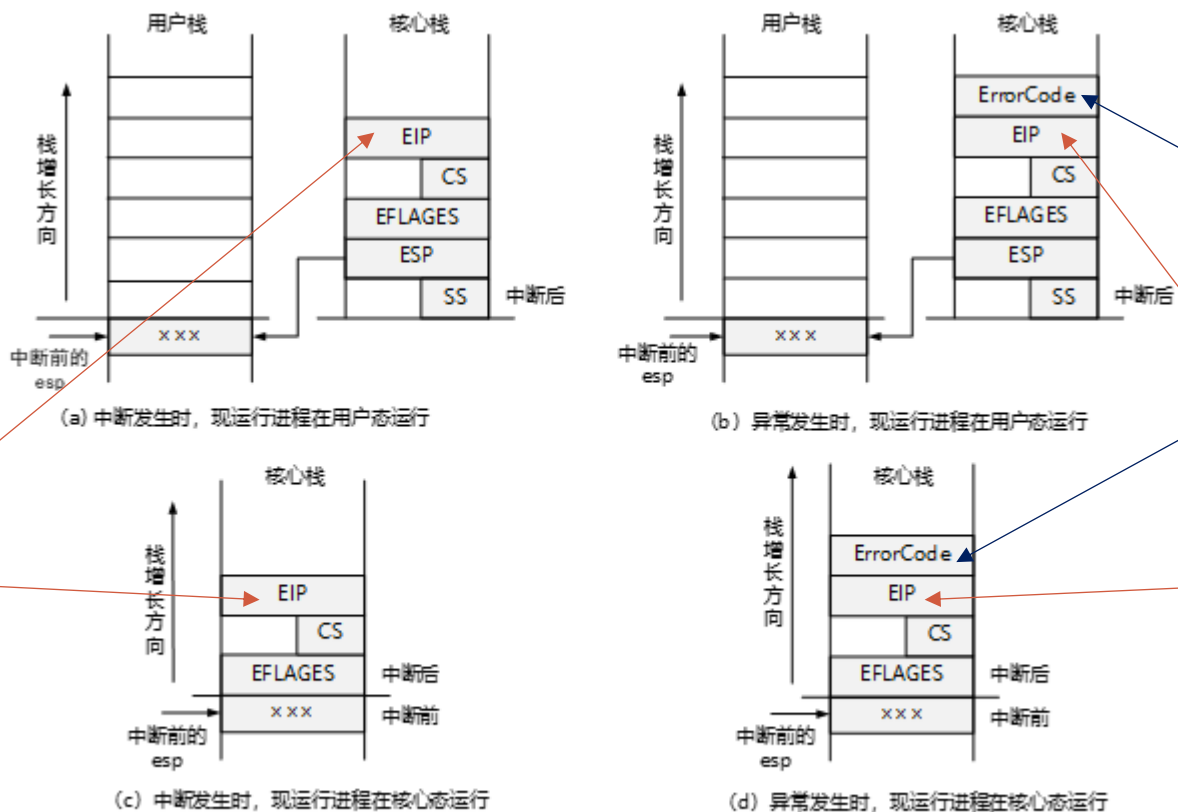
Trap：trap指令（int 3， int 1）抛出的异常。旨在让被debug的程序暂停。异常解除后，被debug的程序从trap指令之后恢复运行。

About：无法修复，现运行进程终止

Table 5-1. Protected-Mode Exceptions and Interrupts

Vector No.	Mne-monic	Description	Type	Error Code	Source
0	#DE	Divide Error	Fault	No	DIV and IDIV instructions. 除法指令
1	#DB	Debug	Fault/ Trap	No	Any code or data reference or the INT 1 instruction.
2	—	NMI Interrupt	Interrupt	No	Nonmaskable external interrupt.
3	#BP	Breakpoint	Trap	No	INT 3 instruction.
4	#OF	Overflow	Trap	No	INTO instruction.
5	#BR	BOUND Range Exceeded	Fault	No	BOUND instruction.
6	#UD	Invalid Opcode (Undefined Opcode)	Fault	No	UD2 instruction or reserved 非法指令 opcode. ¹
7	#NM	Device Not Available (No Math Coprocessor)	Fault	No	Floating-point or WAIT/FWAIT instruction.
8	#DF	Double Fault	Abort	Yes (Zero)	Any instruction that can generate an exception, an NMI, or an INTR.
9		Coprocessor Segment Overrun (reserved)	Fault	No	Floating-point instruction. ²
10	#TS	Invalid TSS	Fault	Yes	Task switch or TSS access.
11	#NP	Segment Not Present	Fault	Yes	Loading segment registers or accessing system segments.
12	#SS	Stack-Segment Fault	Fault	Yes	Stack operations and SS register loads.
13	#GP	General Protection	Fault	Yes	Any memory reference and other protection checks.
14	#PF	Page Fault	Fault	Yes	Any memory reference. 缺页异常
15	—	(Intel reserved. Do not use.)		No	
16	#MF	Floating-Point Error (Math Fault)	Fault	No	Floating-point or WAIT/FWAIT instruction.
17	#AC	Alignment Check	Fault	Yes (Zero)	Any data reference in memory. ³
18	#MC	Machine Check	Abort	No	Error codes (if any) and source are model dependent. ⁴
19	#XF	Streaming SIMD Extensions	Fault	No	SIMD floating-point instructions ⁵
20-31	—	Intel reserved. Do not use.			
32-255	—	User Defined (Nonreserved) Interrupts	Interrupt		External interrupt or INT <i>n</i> instruction.

中断和异常，第一现场保护区的不同



如果有error code
保存在这里
异常处理结束后，记得pop清除这个元素

如果是fault，
故障指令地址

图 3.10：中断隐指令对堆栈的影响



四、Unix V6++的异常处理流程 0#异常

现运行进程执行C语句 $c = a/b$ & $b == 0$ 。

0#异常，无error code。

```
void Exception:: DivideErrorEntrance()  
{  
    SaveContext();  
  
    SwitchToKernel();  
  
    CallHandler(Exception, DivideError);  
  
    RestoreContext();  
  
    Leave();  
  
    InterruptReturn();  
}  
  
void Exception:: DivideError(struct pt_regs* regs, struct pt_context* context)  
{  
    User& u = Kernel::Instance().GetUser();           // user, 指向现运行进程的user结构  
    Process* current = u.u_procp;                     // current, 指向现运行进程的Process结构  
  
    if ( (context->xcs & USER_MODE) == USER_MODE ) // 异常发生在用户态  
    {  
        current->PSignal(User::SIGFPE); // 向现运行进程发SIGFPE信号  
        if ( current->IsSig() )  
            current->PSig(context); // 信号处理  
    }  
    else // 异常发生在核心态  
    {  
        Utility::Panic("Divide Exception!"); // panic, 系统停摆  
    }  
}
```

u_signal[8] == 0, 进程终止
否则, 返回用户态执行信号处理程序



Unix V6++ 的异常处理流程 14#缺页异常

缺页异常, 有 error code

```
void Exception::PageFaultEntrance()\n{\n    SaveContext();\n    SwitchToKernel();\n    CallHandler(Exception, PageFault);\n    RestoreContext();\n    Leave();\n    __asm__ __volatile__ ("addl $4, %%esp" ::);\n    InterruptReturn();\n}
```

```
void Exception::PageFault(struct pt_regs* regs, struct pte_context* context)\n{\n\n    User& u = Kernel::Instance().GetUser();\n    Process* current = u.u_procp;\n    MemoryDescriptor& md = u.u_MemoryDescriptor;\n\n    unsigned int cr2;\n    __asm__ __volatile__ ("mov %%cr2, %0" : "=r"(cr2));\n\n    if( (context->xcs & USER_MODE) == USER_MODE)\n    {\n        if( 满足堆栈扩展条件 )\n            current->SStack();\n\n        else\n        {\n            Diagnose::Write("Invalid MM access");\n            current -> PSignal(User::SIGSEGV);\n            if ( current->IsSig() )\n                current->PSig( (pt_context *)&context->eip );\n\n        }\n    }\n    else\n        Utility::Panic("Page Fault in Kernel Mode.");\n}
```

五、实验 (Ubuntu 18.04)

除以0，进程就挂了

```
#include <unistd.h>
#include <stdio.h>
int
main(void)
{
    int a,b,c;

    for ( ;; )
    {
        printf("输入被除数\n");
        scanf("%d", &a);

        printf("输入除数\n");
        scanf("%d", &b);

        c=a/b;

        printf("%d / %d = %d\n",a,b,c);
    }
}
```

```
drong@ubuntu: ~/Desktop/ostest
drong@ubuntu:~/Desktop/ostest$ gcc noException.c
drong@ubuntu:~/Desktop/ostest$ ./a.out
输入被除数
10
输入除数
2
10 / 2 = 5
输入被除数
2
输入除数
0
Floating point exception (core dumped)
drong@ubuntu:~/Desktop/ostest$
```

应用程序没有设置信号处理方式：
除以0，进程会挂：
1、CPU抛出0#异常，压栈保存除法指令地址
2、现运行进程陷入内核、执行0#异常处理程序，处理8#信号：
u_signal[8]为0，exit()终止自己



应用程序设置信号处理函数后，除0异常杀不死进程

```
noException.c x divide0.c x ctrlc2.c x exception
#include <signal.h>
#include <unistd.h>
#include <stdio.h>

static void sig_dyzero(int);

int
main(void)
{
    int a,b;
    int c;

    if (signal(SIGFPE, sig_dyzero) == SIG_ERR)
        printf("can't catch divide by zero error!");

    for ( ; ; )
    {
        printf("输入被除数\n");
        scanf("%d", &a);

        printf("输入除数\n");
        scanf("%d", &b);

        c=a/b;
        printf("%d / %d = %d\n",a,b,c);
    }
}
```

u_signal[8] = sig_dyzero

```
void Process::PSig(struct pt_context* pContext)
{
    User& u = Kernel::Instance().GetUser();
    int signal = this->p_sig;
    this->p_sig = 0;

    if ( u.u_signal[signal] != 0 )
        .....
    else
        exit(); // 进程终止
}
```

```
static void
sig_dyzero(int signo)    /* argument is signal number */
{
    printf("Divide by zero!\n");
}
```

应用程序设置了信号处理方式之后，除以0，进程不会挂：

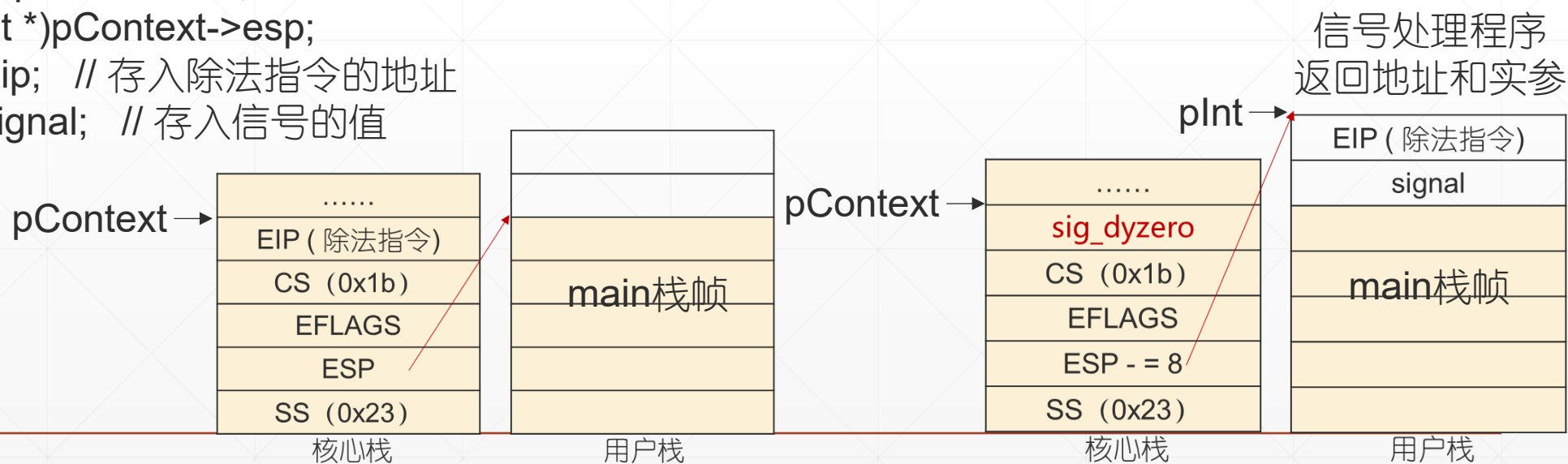
- 1、CPU抛出0#异常，压栈保存除法指令地址
- 2、现运行进程陷入内核、执行0#异常处理程序，处理8#信号：
u_signal[8]不为0，篡改核心栈、用户栈，让进程返回用户态后，先执行信号处理程序。



```
void Process::PSig(struct pt_context* pContext)
{
    User& u = Kernel::Instance().GetUser();
    int signal = this->p_sig;
    this->p_sig = 0;

    if ( u.u_signal[signal] != 0 ) // sig_dzero
    {
        .....
        unsigned int old_eip = pContext->eip; // 除法指令的虚地址
        pContext->eip = u.u_signal[signal]; // 返回用户态, 执行信号处理函数
        pContext->esp -= 8; // 用户栈顶上移 (向低地址端) 8字节
        int* plnt = (int *)pContext->esp;
        *plnt = old_eip; // 存入除法指令的地址
        *(plnt+4) = signal; // 存入信号的值
        .....
        return;
    }
    else
        exit( );
}
```

```
static void
sig_dzero(int signo)      /* argument is signal number */
{
    printf("Divide by zero!\n");
}
```




```
void Process::PSig(struct pt_context* pContext)
```

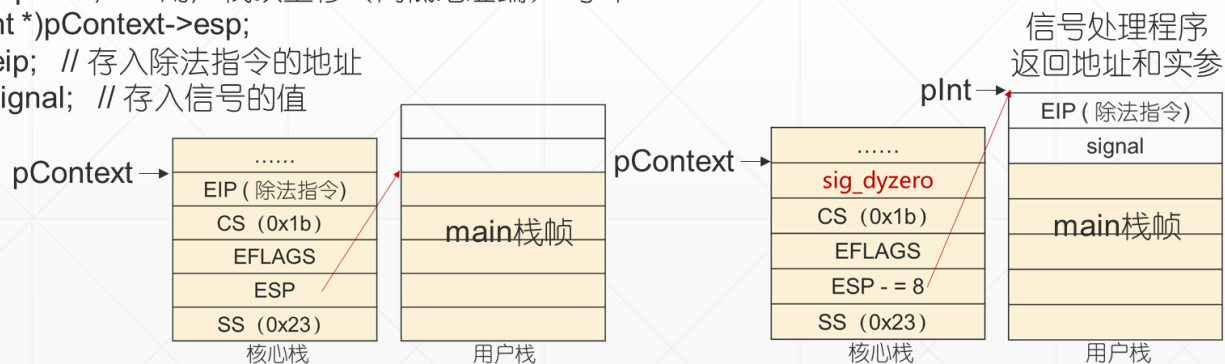
```
{
    User& u = Kernel::Instance().GetUser();
    int signal = this->p_sig;
    this->p_sig = 0;
```

```
    if ( u.u_signal[signal] != 0 ) // sig_dyzero
```

```
    {
        .....
        unsigned int old_eip = pContext->eip; // 除法指令的虚地址
        pContext->eip = u.u_signal[signal]; // 返回用户态，执行信号处理函数
        pContext->esp -= 8; // 用户栈顶上移（向低地址端）8字节
        int* plnt = (int *)pContext->esp;
        *plnt = old_eip; // 存入除法指令的地址
        *(plnt+4) = signal; // 存入信号的值
        .....
        return;
```

```
    }
    else
        exit( );
}
```

```
static void
sig_dyzero(int signo)    /* argument is signal number */
{
    printf("Divide by zero!\n");
}
```



篡改核心栈、用户栈后，0#异常处理完毕。

- 1、恢复用户态寄存器，IRET返回用户态
- 2、执行信号处理程序
- 3、信号处理程序返回后，再次执行除法指令周而复始，无限循环。



```
static void
sig_dyzero(int signo)      /* argument is
{
    printf("Divide by zero!\n");
}
```

操作系统(拔尖班 / 国豪 2025)

参考答案

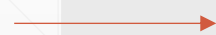
```
static sigjmp_buf fpe_env;
static void sig_dyzero(int);
int
main(void)
{
    int a,b;
    int c;

    if (signal(SIGFPE, sig_dyzero) == SIG_ERR)
        printf("can't catch divide by zero error!");

    for (;;)
    {
        sigsetjmp(fpe_env, 1);

        printf("输入被除数\n");
        scanf("%d", &a);
```

设置一个远程跳转点



```
        printf("输入除数\n");
        scanf("%d", &b);
        c=a/b;
        printf("%d / %d = %d\n",a,b,c);
    }

    static void
    sig_dyzero(int signo)
    {
        printf("Divide by zero!\n");
        siglongjmp(fpe_env, 0);
    }
```

远跳转



附录：异常、信号对照表

#	Exception	Signal
0	Divide error	SIGFPE
1	Debug	SIGTRAP
2	NMI	None
3	Breakpoint	SIGTRAP
4	Overflow	SIGSEGV
5	Bounds check	SIGSEGV
6	Invalid opcode	SIGILL
7	Device not available	SIGSEGV
8	Double fault	SIGSEGV
9	Coprocessor segment overrun	SIGFPE
10	Invalid TSS	SIGSEGV
11	Segment not present	SIGBUS
12	Stack exception	SIGBUS
13	General protection	SIGSEGV
14	Page Fault	SIGSEGV
15	Intel reserved	None
16	Floating-point error	SIGFPE
17	Alignment check	SIGBUS
18	Machine check	None
19	SIMD floating point	SIGFPE

异常处理程序发出的信号



Part 2、外部中断

- 外设中断，表示IO完成。
 - CPU侧，唤醒等待IO完成的进程
 - 外设端，可以接收新的IO命令
- 时钟中断，等间隔脉冲。为计算机内部世界提供“时间”概念
 - 维护系统时钟
 - 调度应用程序，让它们轮流使用CPU
 - 为 内核 和 应用程序 提供定时器



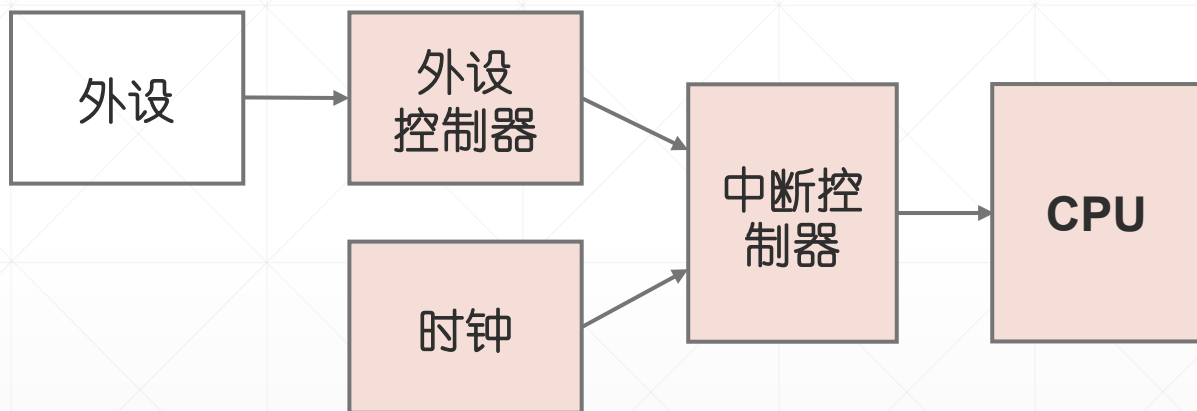
一、计算机系统对外部中断的响应（不考虑并发的中断请求）

- 现运行进程暂停当前执行的任务（应用程序 或 系统调用）
- 陷入内核，在现运行进程的核心栈上执行中断处理程序
- 中断处理结束后，被打断的现运行进程执行中断发生时的下条指令

我们关注的重点在于

- (1) 计算机系统如何与外设交互
- (2) 中断信号对调度子系统的影响
- (3) 怎样确保被中断任务的完整性

二、外部中断子系统架构（硬件）



独立的芯片

外设：键盘、鼠标、显示屏，磁盘、网卡，GPU，以及各种传感器。

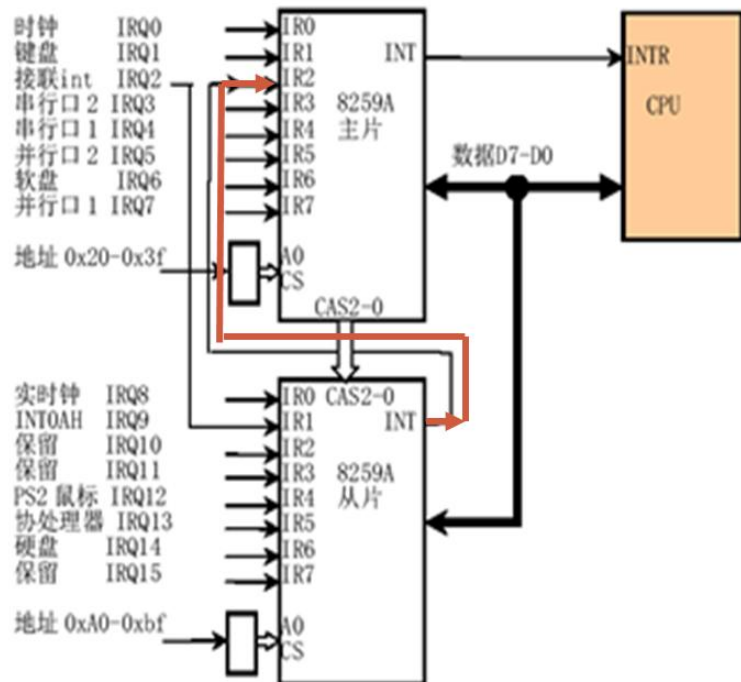
外设控制器：接收CPU发送的IO命令，指挥外设完成IO操作。（1）每次执行一个IO命令（2）IO完成后，发中断请求信号（3）空闲。。。

CPU：设置1个INTR管脚，接收中断请求信号。
∴ 每个时刻，只能响应一个中断请求。

中断控制器：中断仲裁 和 维护。向CPU转发最高优先级中断请求。

中断控制器，PIC（Programmable Interrupt Controller）

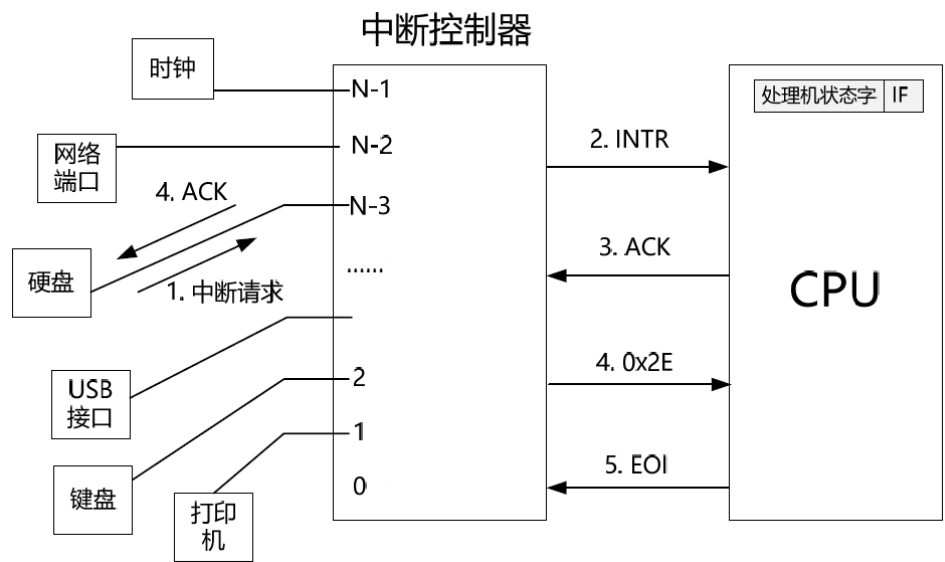
案例：i386中断子系统



- Unix V6++ 系统的中断控制器是2片级联的8259A。
 - 1个INT管脚 连CPU。置高电平，向CPU发中断请求。
 - 很多中断请求线 (irq)，每根线连一个外设，配一个中断向量，有一个中断优先级。
 - 主片 IR0 ~ IR7 (IR2, 连从片的INT)
 - 从片 IR8 ~ IR15
- Unix V6++ 支持 3 种外部中断
 - 时钟挂irq0，中断号0x20。优先级最高。
 - 键盘挂irq1，中断号0x21。优先级最低。
 - 磁盘挂irq14，中断号0x2E。优先级中等。

?

外部中断响应（硬件时序，不考虑中断嵌套）



- 1、IO完成，外设控制器向中断控制器发中断请求
- 2、中断控制器向 CPU 发 中断信号
- 3、若 CPU 开中断，CPU会向中断控制器发ACK信号，表示中断请求已接纳。
 - CPU响应中断，启用核心栈，硬件保护现场（第一现场保护区）
- 4、中断控制器收到ACK信号后
 - 反馈CPU以中断向量（比如磁盘中断，0x2E）。
 - CPU查中断向量表，执行中断入口程序。
 - 向外设控制器发ACK信号，表示中断请求已被系统接纳。
 - 外设控制器收到ACK信号后，撤销中断请求。
 - INT管脚恢复低电平。表示中断响应阶段到此结束

***** CPU 执行中断处理程序，与外设和中断控制器交互 *****

- | | |
|--|--|
| <ol style="list-style-type: none"> 1、接收数据、复位外设芯片 2、如果有新的IO命令，发送给外设 | <ol style="list-style-type: none"> 1、向中断控制器发EOI命令，表示中断请求处理完毕
缺这步，低优先级中断请求永远不会得到响应 |
|--|--|

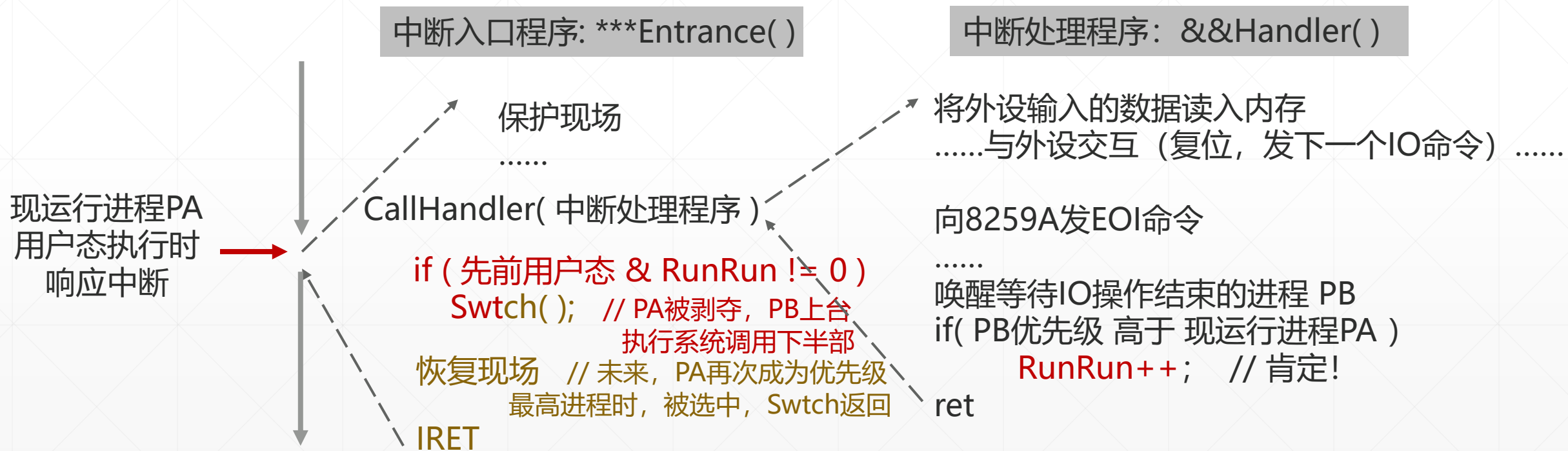
三、外部中断子系统架构（软件）

IDT, 中断描述符表

中断号	语义	门类型	段选择子	offset (中断入口程序首地址)	
外部中断	0x20	时钟中断	中断门	0x8	Time::TimeInterruptEntrance()
	0x21	键盘中断	中断门	0x8	KeyboardInterrupt::KeyboardInterruptEntrance()
	...	...		0x8	IDT::DefaultInterruptHandler()
	0x2E	硬盘中断	中断门	0x8	DiskInterrupt::DiskInterruptEntrance()
	...	...		0x8	IDT::DefaultInterruptHandler()
系统调用	0x80		陷入门	0x8	SystemCall::SystemCallEntrance()

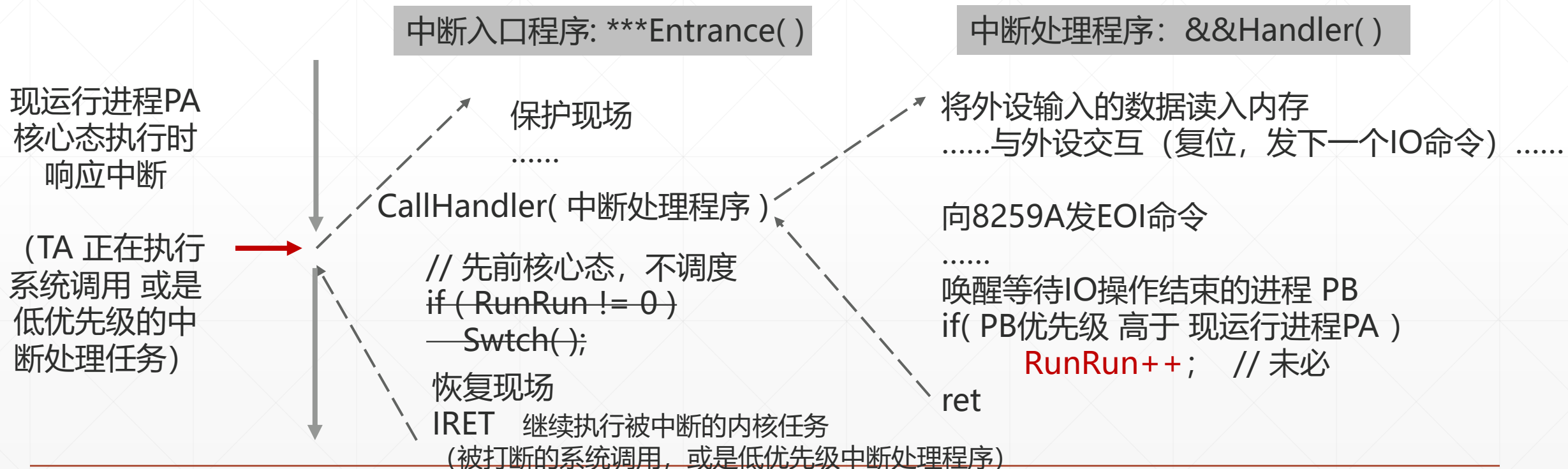
中断处理程序（现运行进程用户态运行，响应外设中断，读操作完成）

- 现运行进程PA响应中断，陷入内核，执行中断处理程序，唤醒等待IO完成的另一个进程，记为PB。返回用户态前，将CPU让给PB，被剥夺（外设中断是IO完成信号）。



中断处理程序（现运行进程核心态运行，响应外设中断，读操作完成）

- 现运行进程PA响应中断，陷入内核，执行中断处理程序，唤醒等待IO完成的另一个进程，记为PB。返回用户态前，将CPU让给PB，被剥夺（外设中断是IO完成信号）。



四、读操作：copy进程执行read系统调用读磁盘文件oldFile



```
int main( int argc, char* argv[] )
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");

    int oldFile = open(argv[1],O_RDONLY);
    int newFile = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR);

    char c;
L: while( read(oldFile,&c,1) == 1 )
        write(newFile,&c,1);

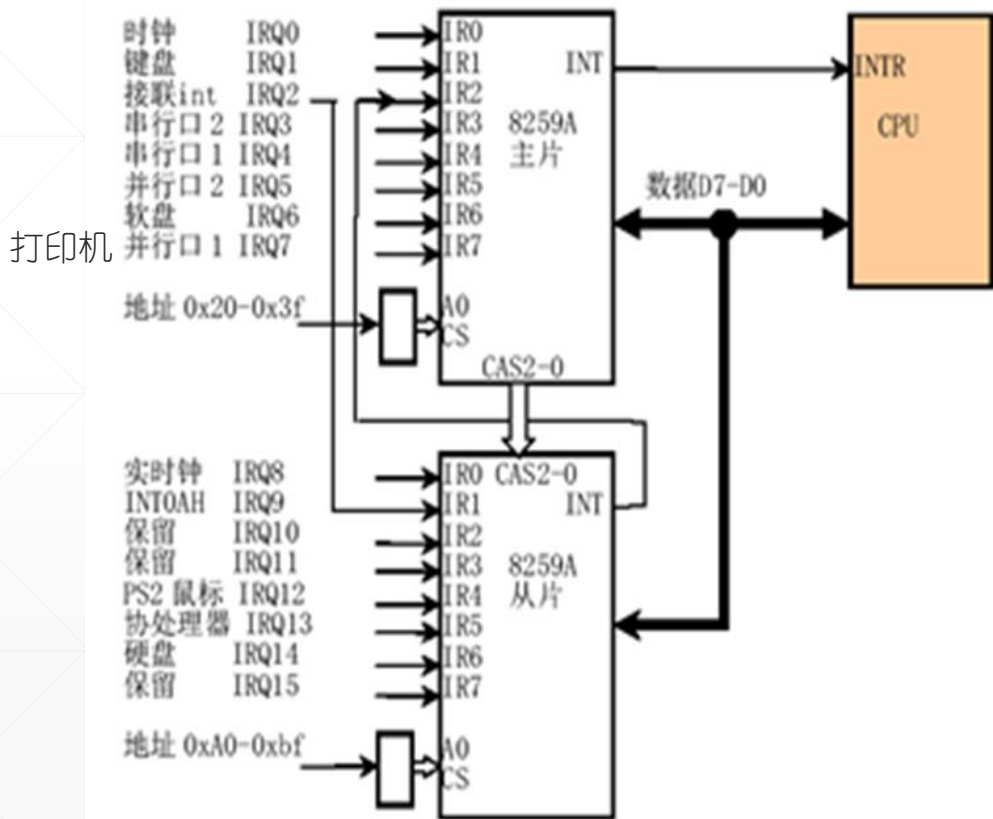
    printf("Copy Done\n");

    close(oldFile);
    close(newFile);
}
```

copy oldFile newFile

- t_0 时刻，copy进程执行**read系统调用上半段**：
时刻 t_1 发磁盘读命令，入睡放弃CPU
[t_1, t_2]，磁盘和DMA控制器为PA服务，
将磁盘上的文件数据读入核心态内存（一个缓存块）
- t_2 时刻，IO完成。被中断的现运行进程PA执行**磁盘中断处理程序**唤醒copy进程。。。
- t_3 时刻，copy进程得到CPU**执行read系统调用下半段**，送数据进用户空间c变量。
- t_4 时刻，copy进程返回用户态执行应用程序。

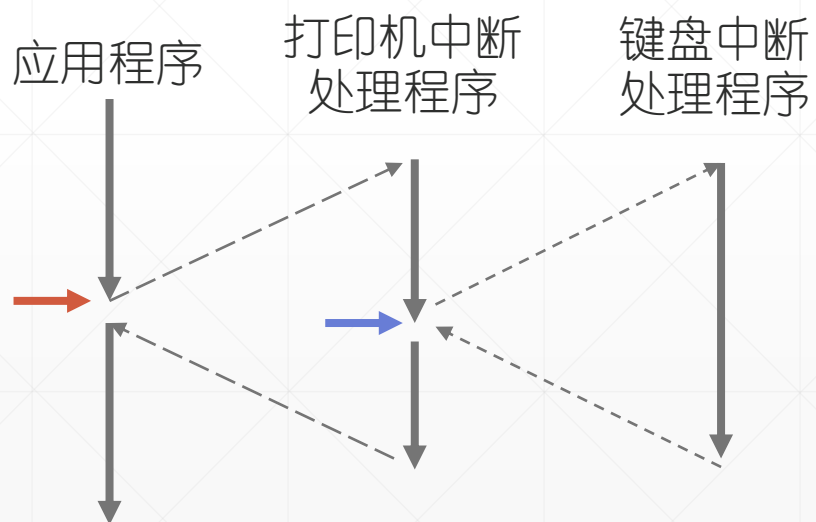
五、并发的中断请求



- 中断请求有优先级。
 - 应用程序和系统调用，中断优先级是0。
 - 外部中断的优先级与IRQ线相关。
- 主片IR0 > 主片IR1 > 从片IR0.....IR7 > 主片IR3.....IR7。
- Unix V6++系统，时钟 > 键盘 > 磁盘
- 优先响应高优先级中断请求。
- 高优先级中断请求可以打断低优先级的中断处理任务。

中断嵌套

- 高优先级中断请求打断正在执行的低优先级中断处理程序



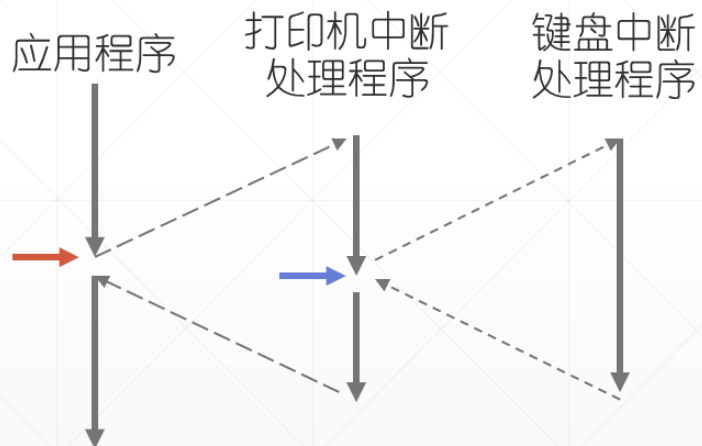
→ T时刻，发生打印机中断请求。响应打印机中断请求，执行打印机中断处理程序。

→ 打印机中断处理程序执行时， $T+\Delta t$ ，发生键盘中断请求。键盘中断请求的优先级高于打印机中断处理程序，响应键盘中断。

键盘中断处理程序执行完毕后，继续执行打印机中断处理程序。

打印机中断处理程序执行完毕后，本次中断嵌套处理过程结束。

中断嵌套时，现场的保护与恢复

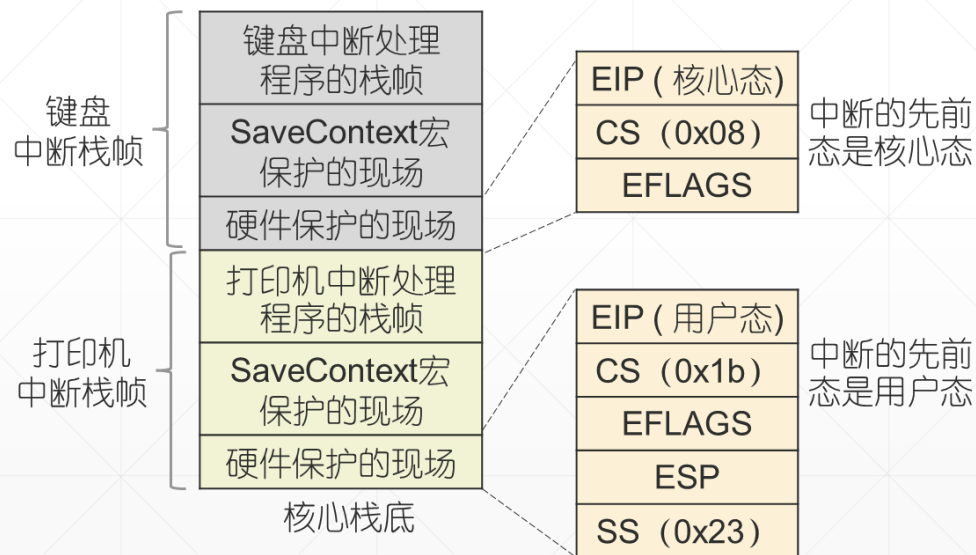


```
void DiskInterrupt::DiskInterruptEntrance()
{
    /* 保存中断现场 */
    SaveContext();
    /* 进入核心态 */
    SwitchToKernel();

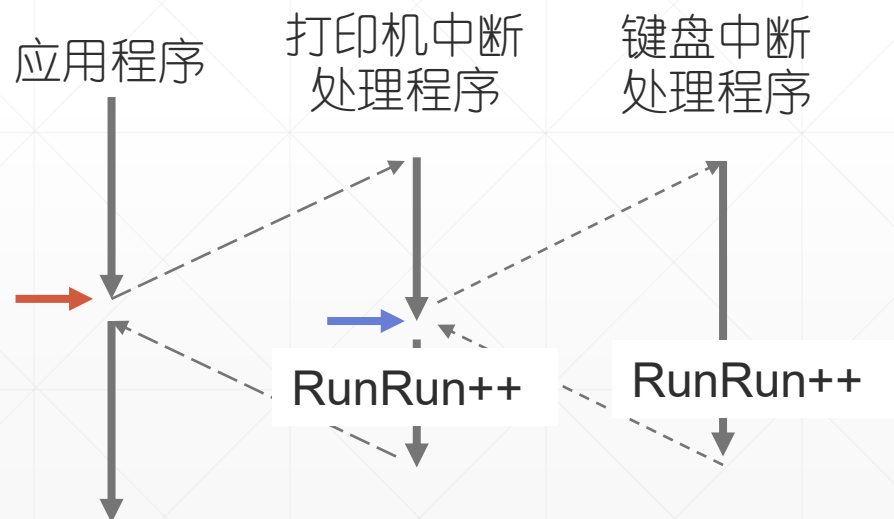
    /* 调用硬盘中断处理子程序 */
    CallHandler(ATADChip8259ADriver, ATAHandler);

    L: /* 例行调度 */
    C代码

    /* 恢复现场 */
    RestoreContext();
    /* 手工销毁栈帧 */
    Leave();
    /* 退出中断 */
    InterruptReturn();
}
```



中断嵌套与调度



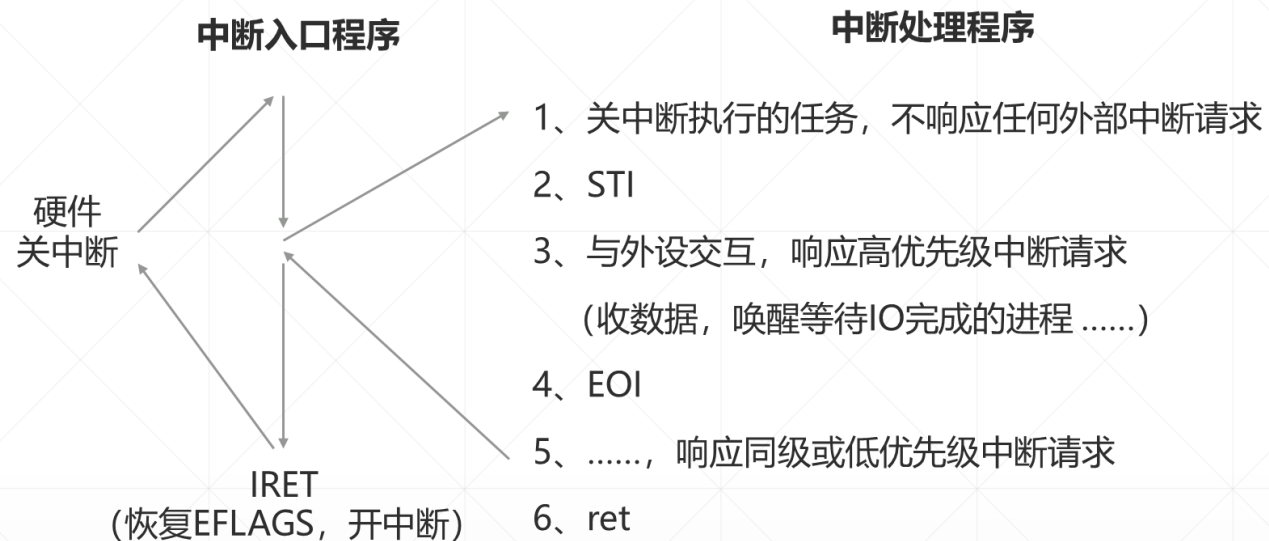
- T时刻，发生打印机中断请求。先前态是用户态，响应中断，发生模式切换，**现运行进程PA**陷入核心态，执行中断处理程序。
- T+Δt，发生键盘中断请求。键盘中断请求的优先级高于CPU正在执行的打印机中断处理程序，发生中断嵌套。先前态是核心态。

键盘中断处理程序**唤醒进程PB**，执行完毕后，IRET。恢复打印机中断处理程序的执行现场，**先前态是核心态，不调度**，PA继续，执行打印机中断处理程序。

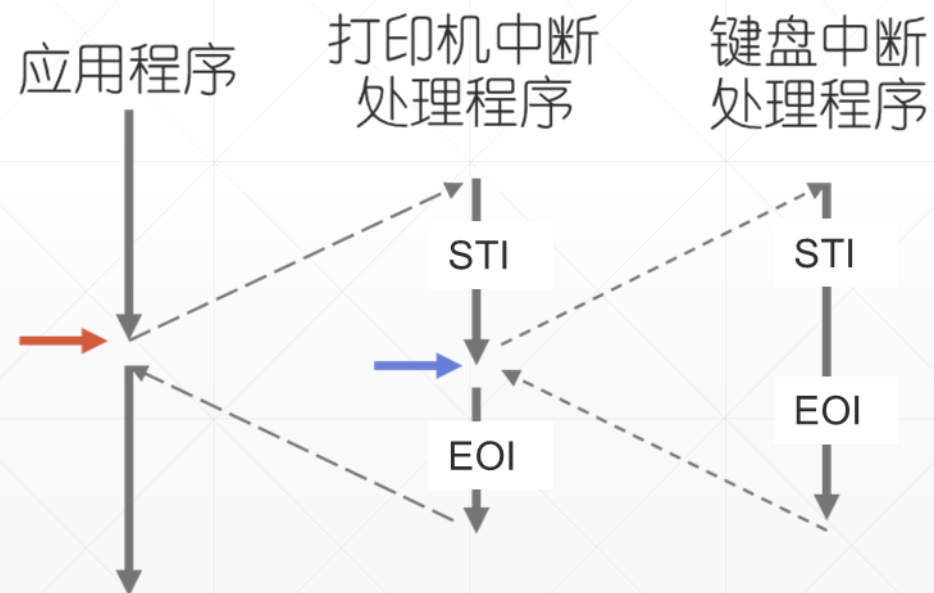
打印机中断处理程序**唤醒进程PC**，执行完毕返回中断入口程序。先前是用户态，例行调度。**RunRun==2，PA放弃CPU**
 优先级高的**PC**先运行（执行优先级高的系统下半段）
 优先级低的**PB**后运行（执行优先级低的系统下半段）
 全部完成后，本次嵌套中断处理结束，**PA、PB、PC**轮流使用**CPU**。

被中断的**PA**进程，待下次被选中运行时，恢复用户态现场，继续执行被中断的应用程序。

细节1：外部中断的中断处理程序



STI后，可以响应低优先级中断请求。
EOI后，可以响应低优先级中断请求。



未被响应的低优先级中断请求

- 低优先级中断请求不能打断高优先级中断请求。
- 高优先级中断请求执行完毕，响应低优先级中断请求。

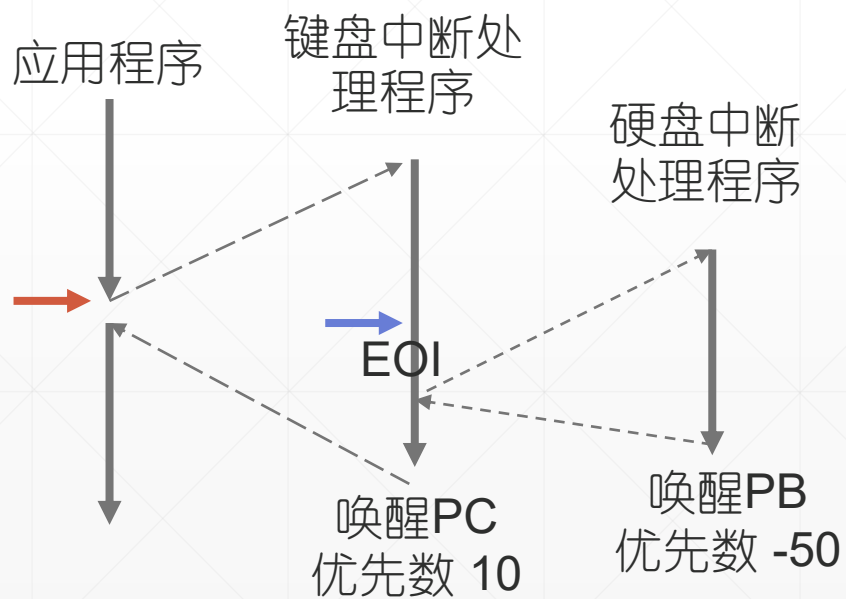
→ T时刻，发生键盘中断请求。先前态是用户态，响应中断，发生模式切换，CPU陷入核心态，执行中断处理程序。

→ T+ Δt ，发生硬盘中断请求。硬盘中断请求的优先级低于CPU正在执行的中断处理程序，不响应。

键盘中断处理执行EOI命令后，立即响应硬盘中断处理请求。

硬盘中断处理程序执行完毕后，继续键盘中断处理过程，跑完以后，先前用户态，例行调度，现运行进程PA放弃CPU。

后面我们会知道，硬盘中断处理程序唤醒的进程PB优先级远远高于键盘中断处理程序唤醒的进程PC，所以PB先跑执行系统调用下半段，完成后PC执行它的下半段。



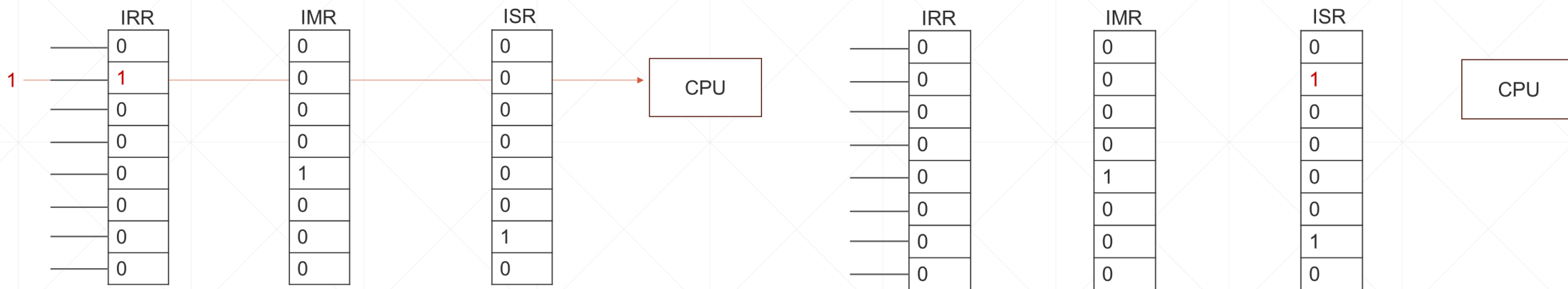
细节 2、8259A的中断管理

8259A硬件

- 中断请求寄存器 IRR, 8个bit。每个bit对应一根中断请求线。
IR0~IR7上出现中断请求信号, 对应 bit 置 1。
- 中断屏蔽寄存器 IMR, 8个bit。内核暂时不希望处理这个中断请求时, 对应bit置1。
- 中断服务寄存器 ISR, 8个bit。CPU正在服务的中断请求, 对应bit为1。
中断嵌套时, 会有多个 1 bit。
- 中断向量寄存器 IVR, 送给CPU的中断号。也就是CPU正在执行的中断处理程序的中断号。

8259A的中断转发

中断请求寄存器 IRR
中断屏蔽寄存器 IMR
中断服务寄存器 ISR
中断向量寄存器 IVR



irq线request(1)上出现中断请求

IRR[request] = 1
if(IMR[request] == 0 & IVR₍₆₎ > request₍₂₎)
向CPU发中断请求

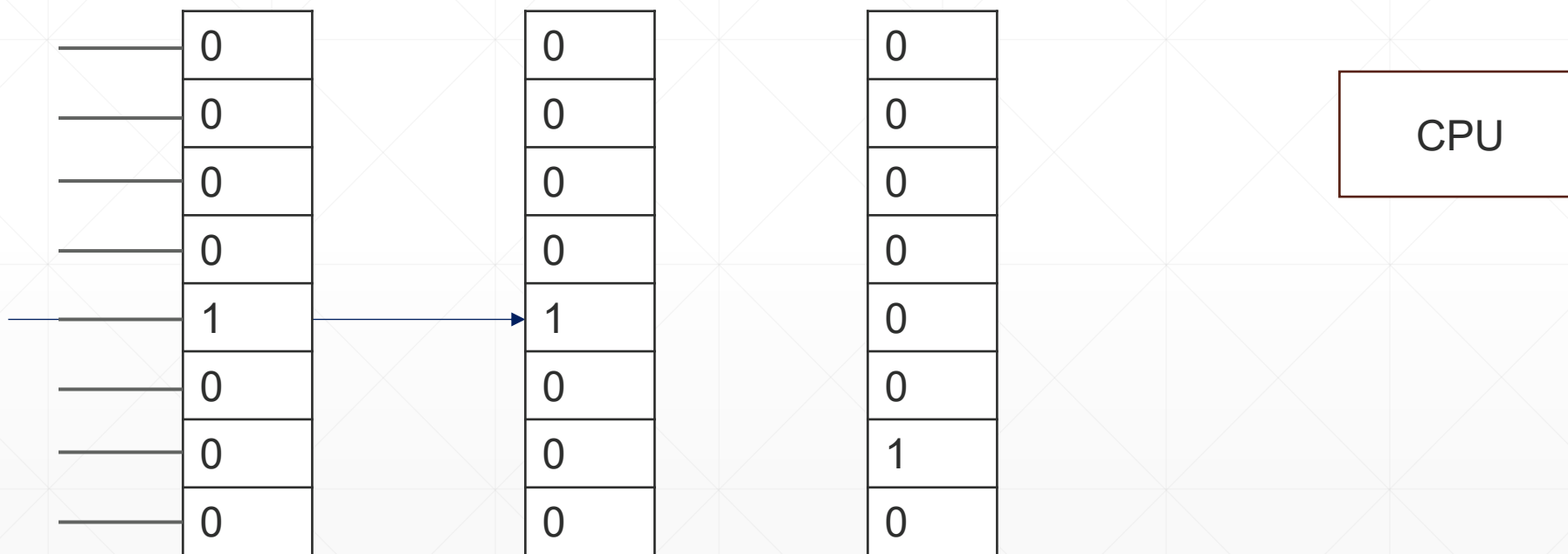
CPU执行完当前指令后，若IF==1，响应中断请求

CPU向8259A发
第1个INTA信号

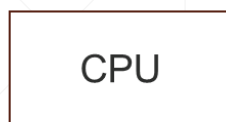
CPU发第2个INTA信
号给8259A，读IVR。
执行中断处理程序

8259A，IRR中找下标最小的1bit
IVR=下标 (1) 。
ISR[request] = 1。
IRR[request] = 0。// 该中断请求得到处理

中断请求被屏蔽



EOI命令 (End Of Interrupt)



中断处理程序
{
 中断处理任务
 EOI
}

执行完中断处理任务后，中断处理程序向8259A发EOI命令。

8259A，收到命令后

$ISR[IVR] = 0$

ISR中下标最小的1bit，将下标赋给IVR。若找不到， $IVR = 8$ 。

if(存在 $IRR[request] == 1$ & $IMR[request] == 0$ & $IVR > request$)
 向CPU发中断请求request